telco-churn-eda

March 23, 2026

0.1 Telco Churn Analysis

Dataset info : Sample dataset containing Telco customer data and showing customers left last month

[3]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.ticker as mtick
import matplotlib.pyplot as plt
%matplotlib inline
import plotly.express as px
```

Importing data

[5]:
```
data = pd.read_csv("F:\\data sets\\WA_Fn-UseC_-Telco-Customer-Churn.csv")
data.head()
```

[5]:
```
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  … DeviceProtection  \
0  No phone service             DSL             No  …               No
1                No             DSL            Yes  …              Yes
2                No             DSL            Yes  …               No
3  No phone service             DSL            Yes  …              Yes
4                No     Fiber optic             No  …               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes
```

```
                PaymentMethod MonthlyCharges  TotalCharges Churn
0            Electronic check          29.85         29.85    No
1                Mailed check          56.95        1889.5    No
2                Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4            Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
```

Cheking the various attributes of data like shape(rows and cols),Columns,datatypes

[7]:
```
data.shape ## we have 7043-> columns and 21->rows
```

[7]:
```
(7043, 21)
```

[8]:
```
data.columns
```

[8]:
```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='str')
```

[9]:
```
data.dtypes
```

[9]:
```
customerID              str
gender                  str
SeniorCitizen         int64
Partner                 str
Dependents              str
tenure                int64
PhoneService            str
MultipleLines           str
InternetService         str
OnlineSecurity          str
OnlineBackup            str
DeviceProtection        str
TechSupport             str
StreamingTV             str
StreamingMovies         str
Contract                str
PaperlessBilling        str
PaymentMethod           str
MonthlyCharges      float64
TotalCharges            str
Churn                   str
```

```
dtype: object
```

Checking the descriptive statistics of numeric variables

[11]:
```
data.describe()
```

[11]:
```
       SeniorCitizen       tenure  MonthlyCharges
count    7043.000000  7043.000000     7043.000000
mean        0.162147    32.371149       64.761692
std         0.368612    24.559481       30.090047
min         0.000000     0.000000       18.250000
25%         0.000000     9.000000       35.500000
50%         0.000000    29.000000       70.350000
75%         0.000000    55.000000       89.850000
max         1.000000    72.000000      118.750000
```

SeniorCitizen is actually a categorical hence the 25%-50%-75% distribution is not proper

75% customers have tenure less than 55 months, tenure means number of months a customer has stayed with the company

Average Monthly charges are USD 64.76 whereas 25% customers pay more than USD 89.85 per month

[15]:
```
churn_count = data['Churn'].value_counts()
churn_count
```

[15]:
```
Churn
No     5174
Yes    1869
Name: count, dtype: int64
```

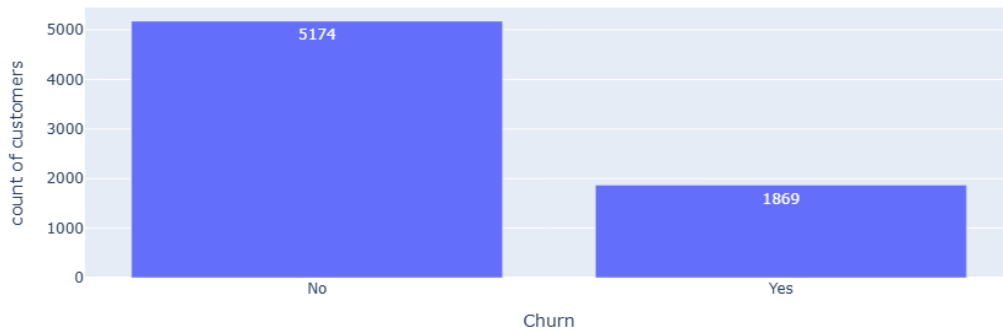
[16]:
```
(data['Churn'].value_counts()/len(data['Churn']))*100
```

[16]:
```
Churn
No     73.463013
Yes    26.536987
Name: count, dtype: float64
```

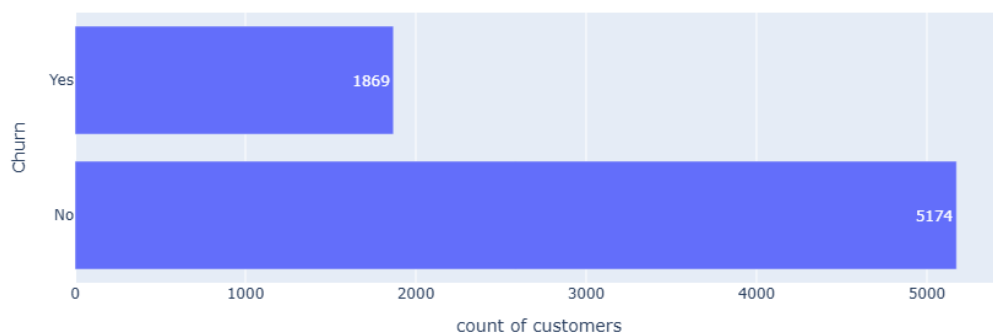
Data is highly imbalanced, ratio = 73:27 leads to overfitting

So we analyse the data with other features while taking the target values separately to get some insights

[19]:
```
px.bar(churn_count,x=churn_count.index,y=churn_count.values,text=churn_count.
 ↪values,labels={'y':'count of customers'})
```



[20]:
```
px.bar(churn_count,x=churn_count.values,y=churn_count.index,text=churn_count.
↪values,labels={'x':'count of customers'},orientation='h')
```



[21]:
```
data.info(verbose=True)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   str
 1   gender            7043 non-null   str
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   str
 4   Dependents        7043 non-null   str
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   str
```

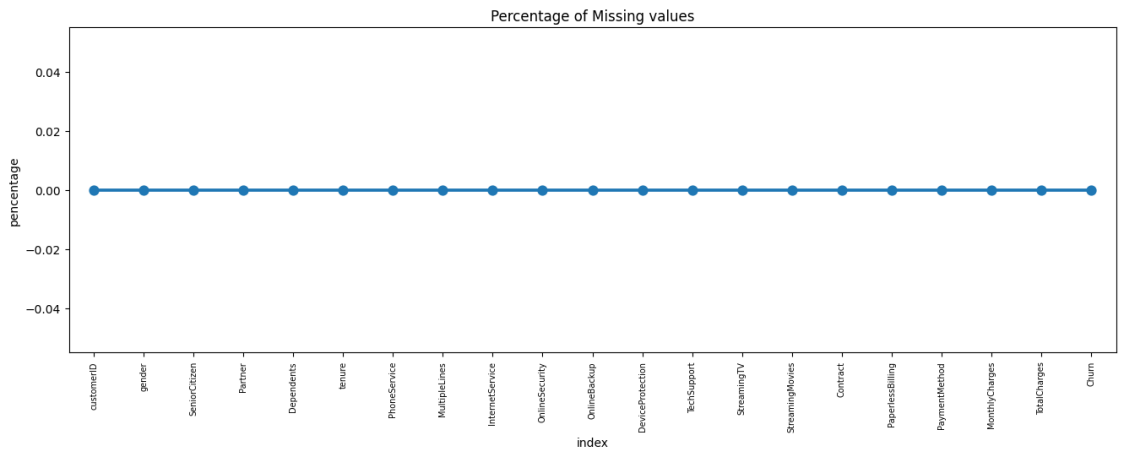
```
 7   MultipleLines     7043 non-null   str
 8   InternetService   7043 non-null   str
 9   OnlineSecurity    7043 non-null   str
 10  OnlineBackup      7043 non-null   str
 11  DeviceProtection  7043 non-null   str
 12  TechSupport       7043 non-null   str
 13  StreamingTV       7043 non-null   str
 14  StreamingMovies   7043 non-null   str
 15  Contract          7043 non-null   str
 16  PaperlessBilling  7043 non-null   str
 17  PaymentMethod     7043 non-null   str
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   str
 20  Churn             7043 non-null   str
dtypes: float64(1), int64(2), str(18)
memory usage: 1.9 MB
```

[22]:
```
missing_data = pd.DataFrame((data.isnull().sum())*100/len(data)).reset_index()
missing_data
```

[22]:
```
                index    0
0          customerID  0.0
1              gender  0.0
2       SeniorCitizen  0.0
3             Partner  0.0
4          Dependents  0.0
5              tenure  0.0
6        PhoneService  0.0
7       MultipleLines  0.0
8     InternetService  0.0
9      OnlineSecurity  0.0
10       OnlineBackup  0.0
11   DeviceProtection  0.0
12       TechSupport  0.0
13       StreamingTV  0.0
14   StreamingMovies  0.0
15          Contract  0.0
16  PaperlessBilling  0.0
17     PaymentMethod  0.0
18    MonthlyCharges  0.0
19      TotalCharges  0.0
20             Churn  0.0
```

[23]:
```
plt.figure(figsize=(16,5))
ax = sns.pointplot(x='index',y=0,data=missing_data)
plt.xticks(rotation=90, fontsize=7)
plt.title("Percentage of Missing values")
```

```
plt.ylabel("pencentage")
plt.show()
```



There are no missing data

1 Data Cleaning

1. Created a copy of base data for manupulation & processing

[27]:
```
telco_data = data.copy()
```

2. Total Charges is in str datatype , Let's convert it to numerical data type

[29]:
```
telco_data.TotalCharges = pd.to_numeric(telco_data.TotalCharges,␣
↪errors='coerce')
# telco_data['TotalCharges'] = telco_data['TotalCharges'].astype(float)
print(telco_data.TotalCharges.isnull().sum())
```

```
11
```

3 . we can see there are 11 missing values in TotalCharges column . Let's check these records

[31]:
```
telco_data.loc[telco_data['TotalCharges'].isnull()==True]
```

[31]:
```
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
488   4472-LVYGI  Female              0     Yes        Yes       0
753   3115-CZMZD    Male              0      No        Yes       0
936   5709-LVOEQ  Female              0     Yes        Yes       0
1082  4367-NUYAO    Male              0     Yes        Yes       0
1340  1371-DWPAZ  Female              0     Yes        Yes       0
3331  7644-OMVMY    Male              0     Yes        Yes       0
3826  3213-VVOLG    Male              0     Yes        Yes       0
```

4380	2520-SGTTA	Female	0	Yes	Yes	0
5218	2923-ARZLG	Male	0	Yes	Yes	0
6670	4075-WKNIU	Female	0	Yes	Yes	0
6754	2775-SEFEE	Male	0	No	Yes	0

	PhoneService	MultipleLines	InternetService	OnlineSecurity	…	\
488	No	No phone service	DSL	Yes	…	
753	Yes	No	No	No internet service	…	
936	Yes	No	DSL	Yes	…	
1082	Yes	Yes	No	No internet service	…	
1340	No	No phone service	DSL	Yes	…	
3331	Yes	No	No	No internet service	…	
3826	Yes	Yes	No	No internet service	…	
4380	Yes	No	No	No internet service	…	
5218	Yes	No	No	No internet service	…	
6670	Yes	Yes	DSL	No	…	
6754	Yes	Yes	DSL	Yes	…	

	DeviceProtection	TechSupport	StreamingTV	\
488	Yes	Yes	Yes	
753	No internet service	No internet service	No internet service	
936	Yes	No	Yes	
1082	No internet service	No internet service	No internet service	
1340	Yes	Yes	Yes	
3331	No internet service	No internet service	No internet service	
3826	No internet service	No internet service	No internet service	
4380	No internet service	No internet service	No internet service	
5218	No internet service	No internet service	No internet service	
6670	Yes	Yes	Yes	
6754	No	Yes	No	

	StreamingMovies	Contract	PaperlessBilling	\
488	No	Two year	Yes	
753	No internet service	Two year	No	
936	Yes	Two year	No	
1082	No internet service	Two year	No	
1340	No	Two year	No	
3331	No internet service	Two year	No	
3826	No internet service	Two year	No	
4380	No internet service	Two year	No	
5218	No internet service	One year	Yes	
6670	No	Two year	No	
6754	No	Two year	Yes	

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
488	Bank transfer (automatic)	52.55	NaN	No
753	Mailed check	20.25	NaN	No

```
936                Mailed check          80.85           NaN     No
1082               Mailed check          25.75           NaN     No
1340    Credit card (automatic)          56.05           NaN     No
3331               Mailed check          19.85           NaN     No
3826               Mailed check          25.35           NaN     No
4380               Mailed check          20.00           NaN     No
5218               Mailed check          19.70           NaN     No
6670               Mailed check          73.35           NaN     No
6754  Bank transfer (automatic)          61.90           NaN     No

[11 rows x 21 columns]
```

4 . Missing value Treatment

since the % of missing values compared to total dataset is very low ie 0.15%, it is safe to ignore them from further processing

[34]:
```
# removing missing values
telco_data.dropna(how = 'any',inplace = True)
```

5. Divide Customer into bins

ex - for tenure <12 months assign a tenure group of 1-12, for tenure between 1 to 2 yrs,tenure group of 13-24; so on

[37]:
```
# get the max tenure
print(telco_data['tenure'].max())
```

```
72
```

[38]:
```
arr = telco_data['tenure'].values
unique_val = np.unique(arr)
print(unique_val)
print(len(unique_val)//12)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48
 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72]
6
```

[39]:
```
# Grouping the tenure in bins of 12 months
labels = ['{0}-{1}'.format(i,i+11) for i in range(1,72,12)]
telco_data['tenure_group']=pd.cut(telco_data.tenure,bins= range(1,80,12),␣
 ↪right=False, labels = labels)
```

[40]:
```
telco_data['tenure_group'].value_counts()
```

[40]:
```
tenure_group
1-12     2175
```

```
61-72    1407
13-24    1024
25-36     832
49-60     832
37-48     762
Name: count, dtype: int64
```

6. Removed columns which are not required for processing

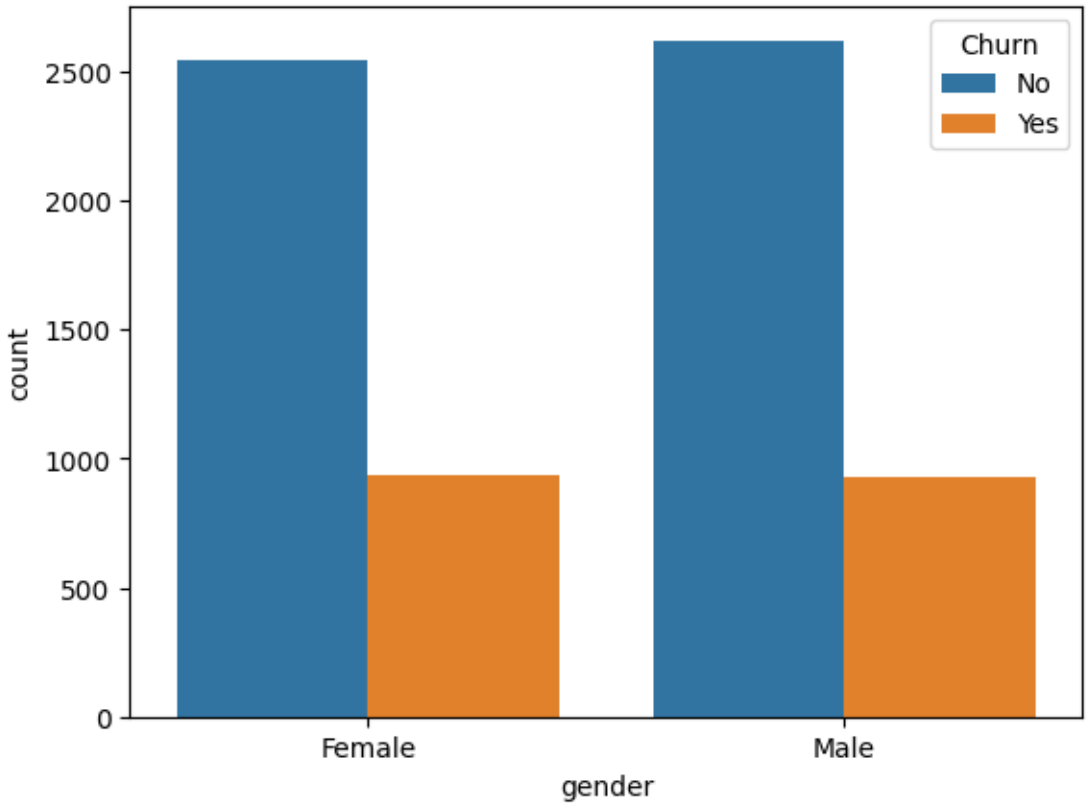
[42]:
```python
# droped column customerID and tenure
telco_data.drop(columns=['customerID','tenure'], inplace=True)
telco_data.head()
```

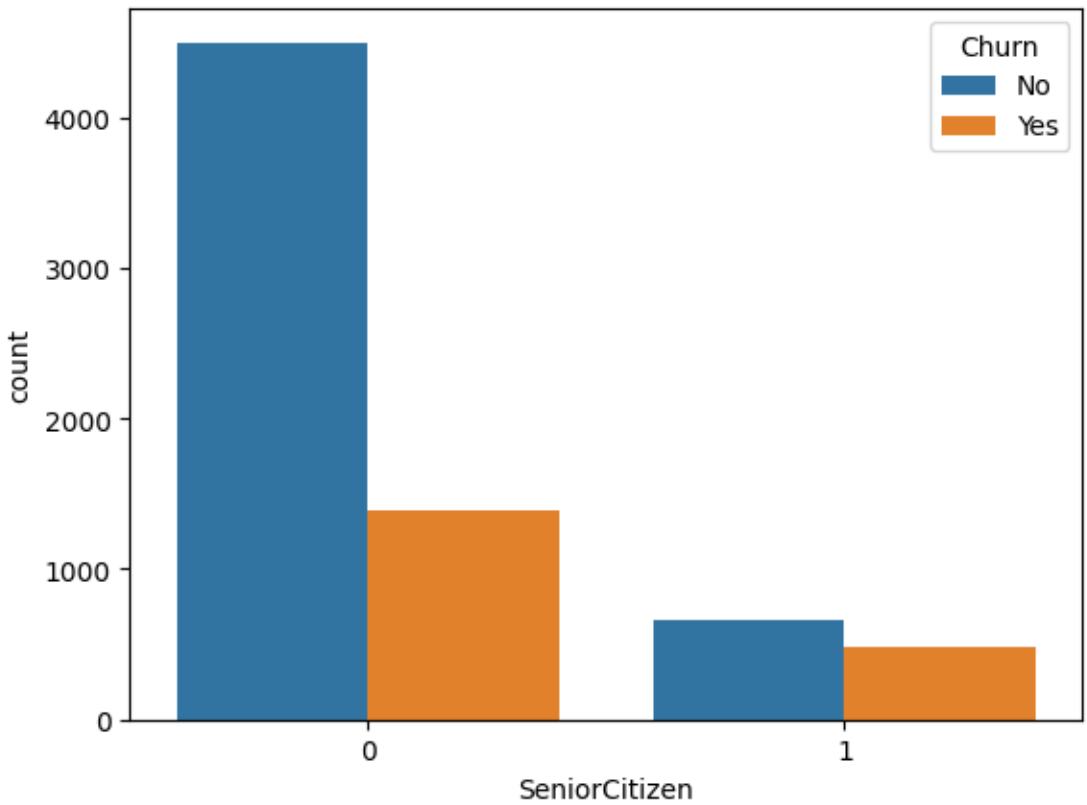
[42]:
```
   gender  SeniorCitizen Partner Dependents PhoneService     MultipleLines  \
0  Female              0     Yes         No           No  No phone service
1    Male              0      No         No          Yes                No
2    Male              0      No         No          Yes                No
3    Male              0      No         No           No  No phone service
4  Female              0      No         No          Yes                No

  InternetService OnlineSecurity OnlineBackup DeviceProtection TechSupport  \
0             DSL             No          Yes               No          No
1             DSL            Yes           No              Yes          No
2             DSL            Yes          Yes               No          No
3             DSL            Yes           No              Yes         Yes
4     Fiber optic             No           No               No          No

  StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No              No  Month-to-month              Yes
1          No              No        One year               No
2          No              No  Month-to-month              Yes
3          No              No        One year               No
4          No              No  Month-to-month              Yes

               PaymentMethod  MonthlyCharges  TotalCharges Churn tenure_group
0           Electronic check           29.85         29.85    No         1-12
1               Mailed check           56.95       1889.50    No        25-36
2               Mailed check           53.85        108.15   Yes         1-12
3  Bank transfer (automatic)           42.30       1840.75    No        37-48
4           Electronic check           70.70        151.65   Yes         1-12
```

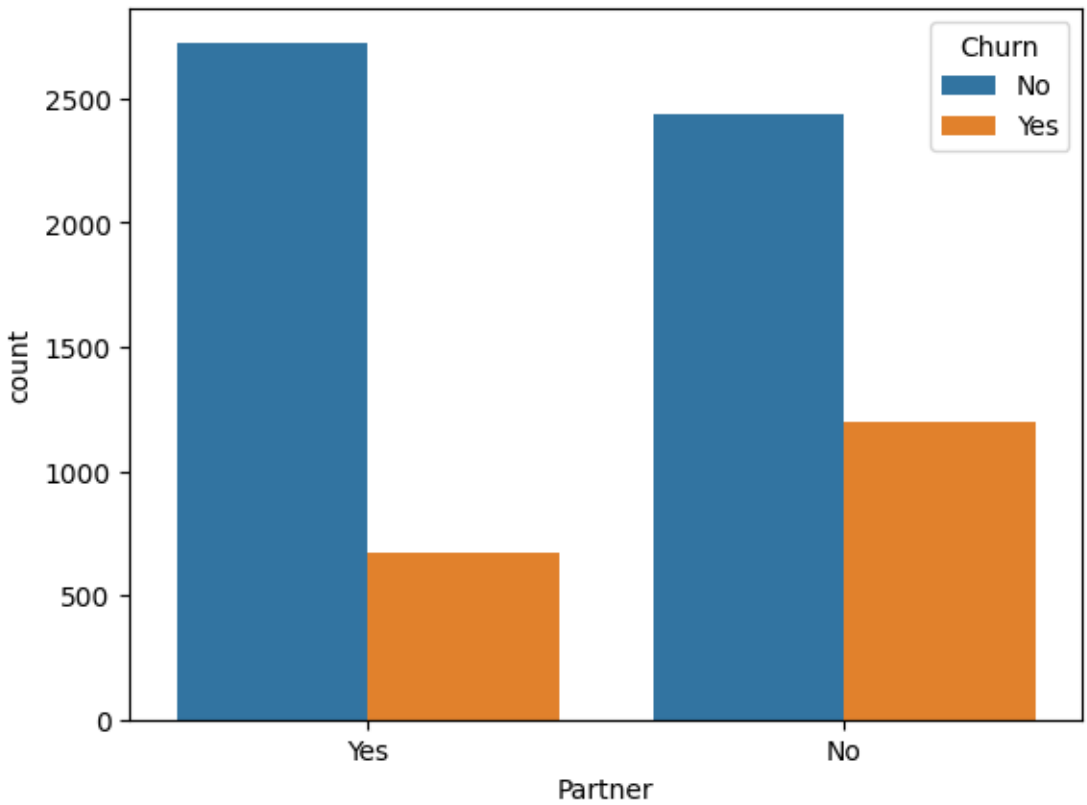
2 Data Exploration

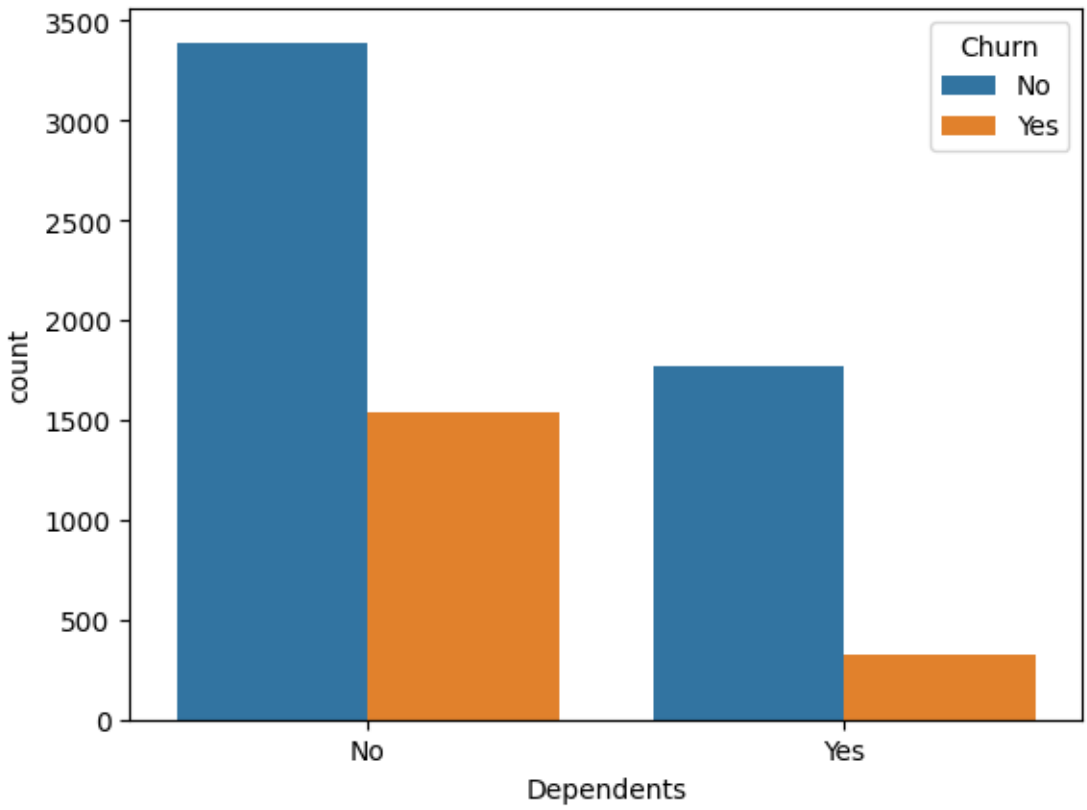
2.0.1 Univariate Analysis

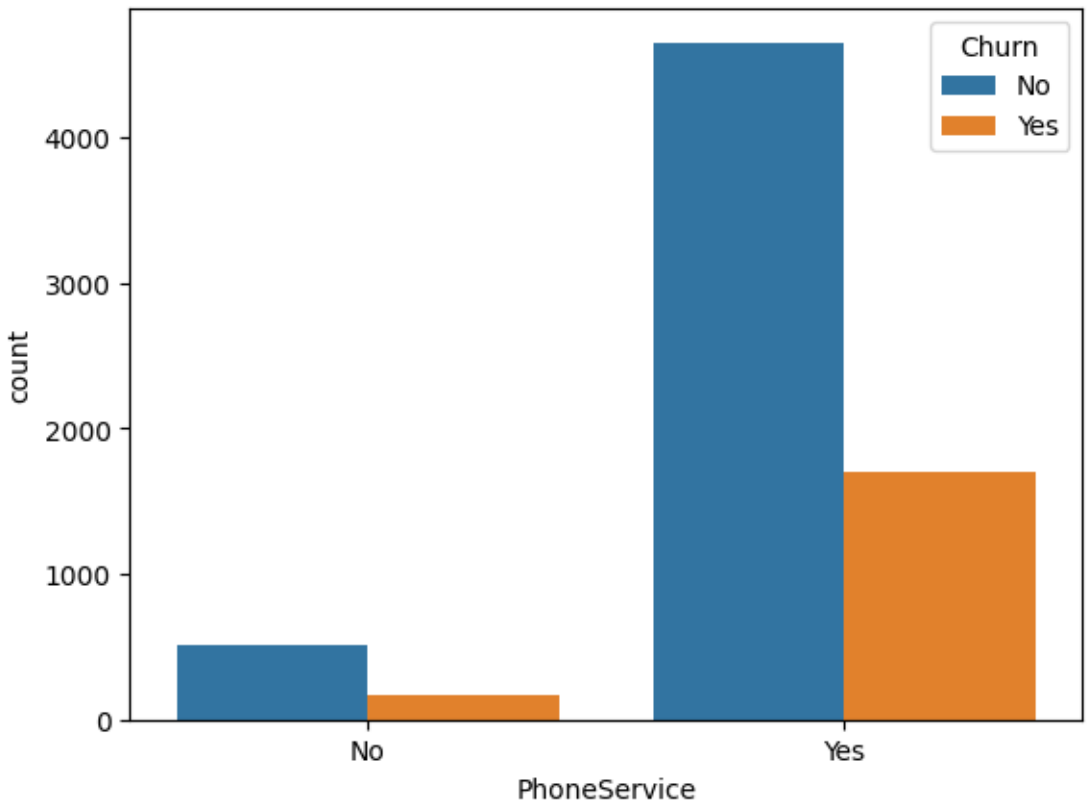
[45]:
```
for i , predictor in enumerate(telco_data.
↪drop(columns=['Churn','TotalCharges','MonthlyCharges'])):
    plt.figure(i)
    sns.countplot(data=telco_data, x=predictor, hue='Churn')
    plt.show()
```

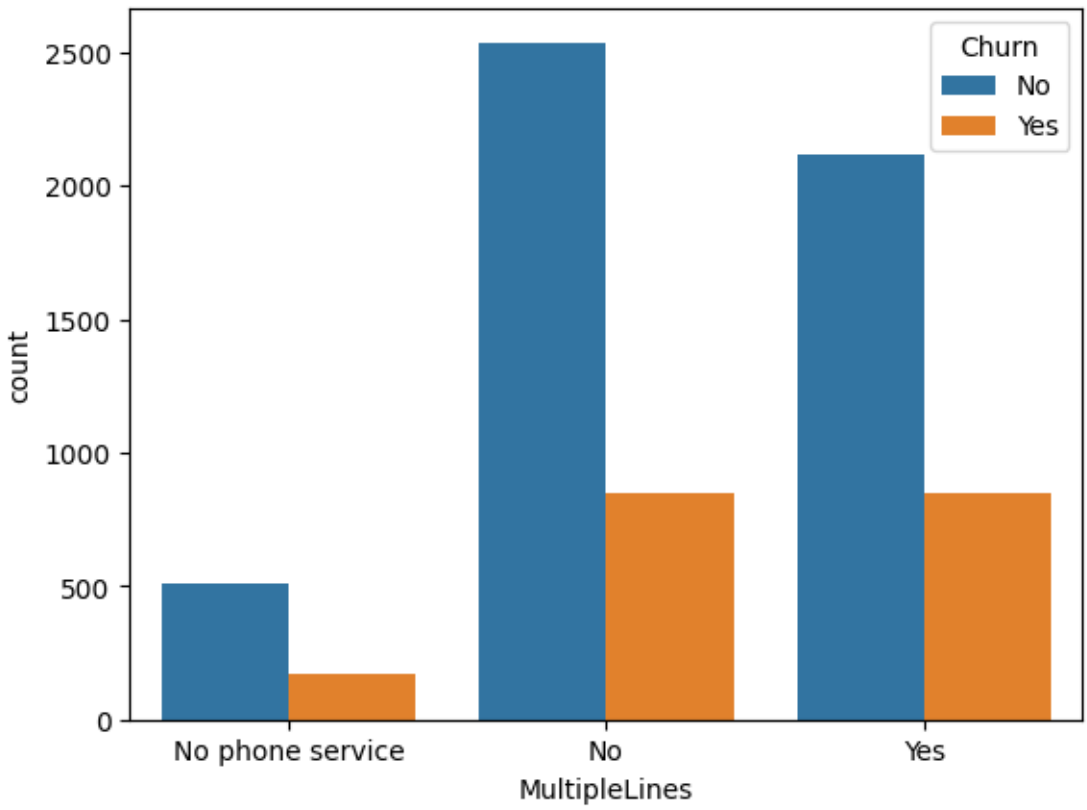


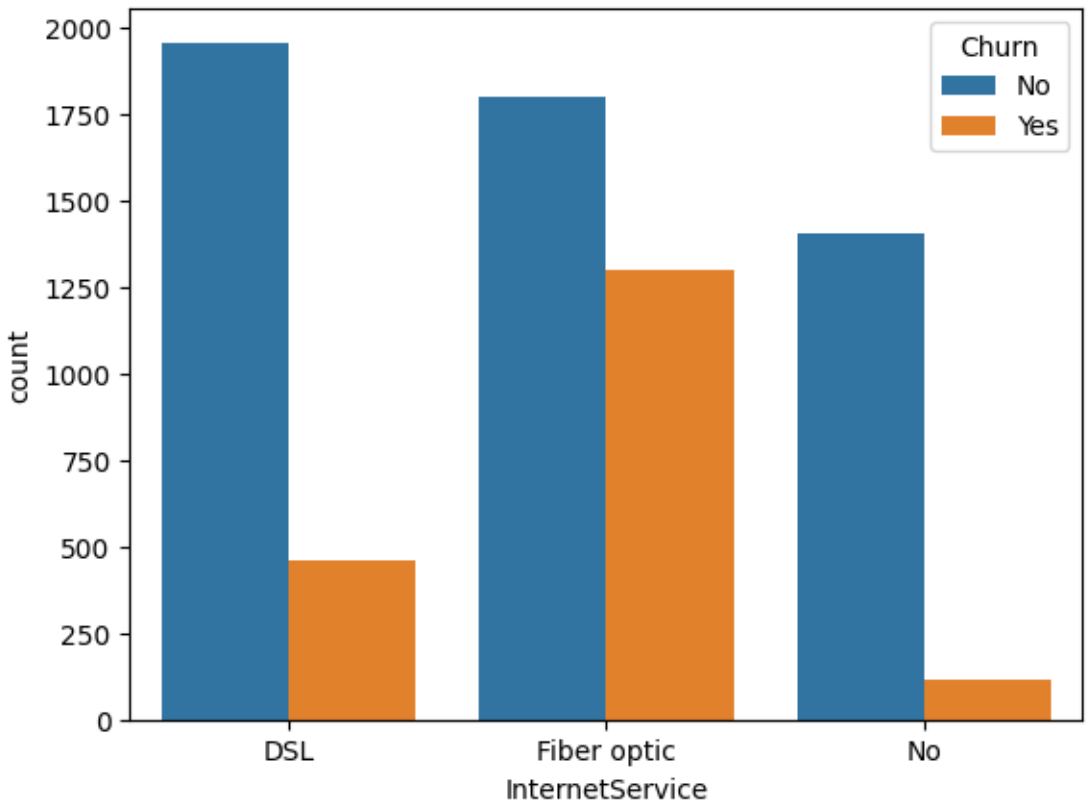

Churn
No
Yes
4000
3000
2000
1000
0
count
0
1
SeniorCitizen

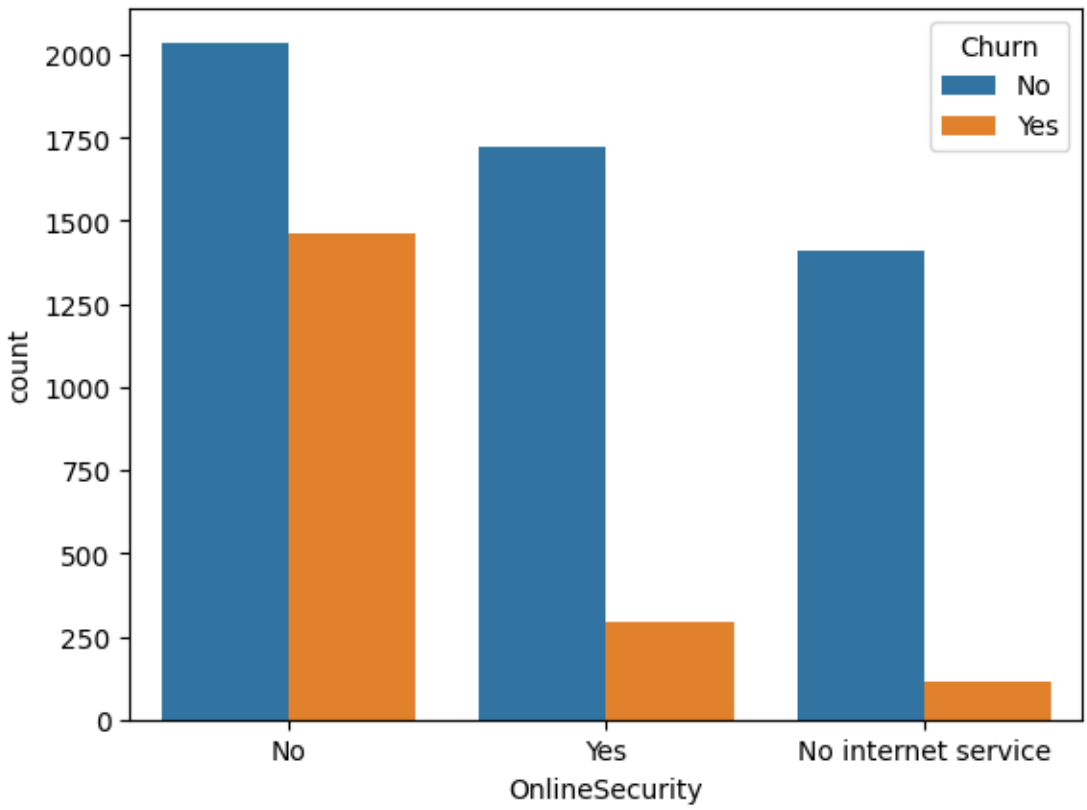

Churn
No
Yes
2500
2000
1500
1000
500
0
count
Yes
No
Partner

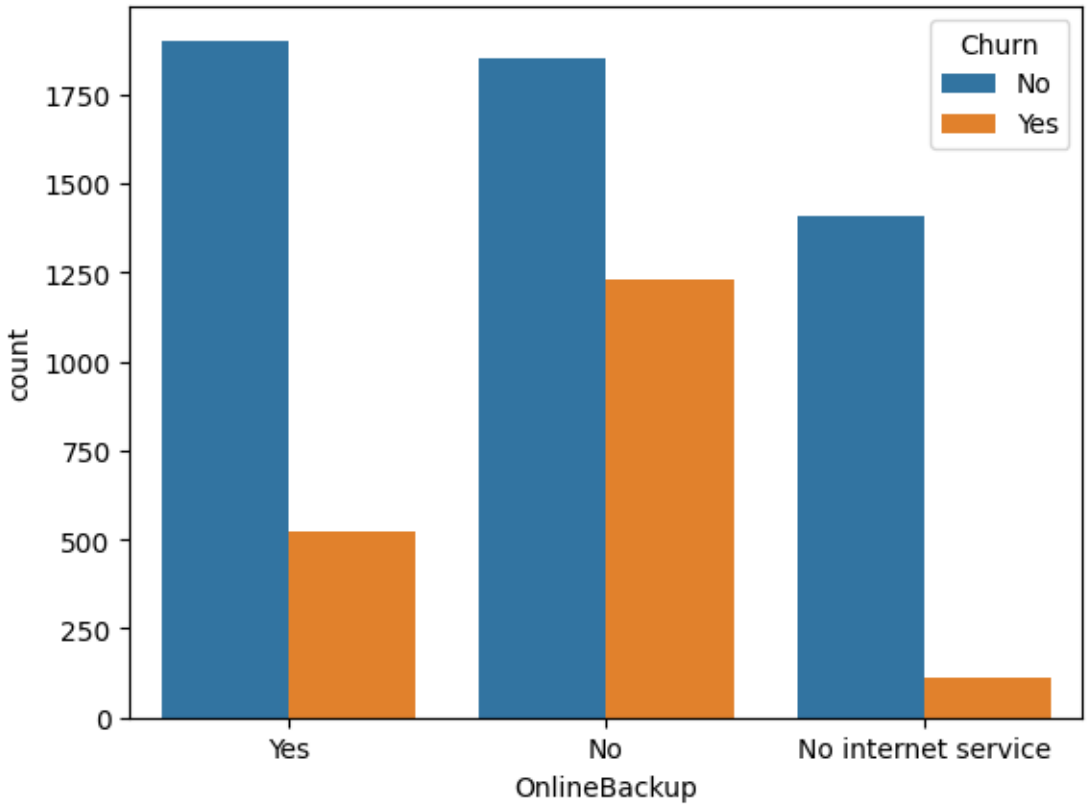

3500
3000
2500
2000
1500
1000
500
0
count
Churn
No
Yes
No
Yes
Dependents

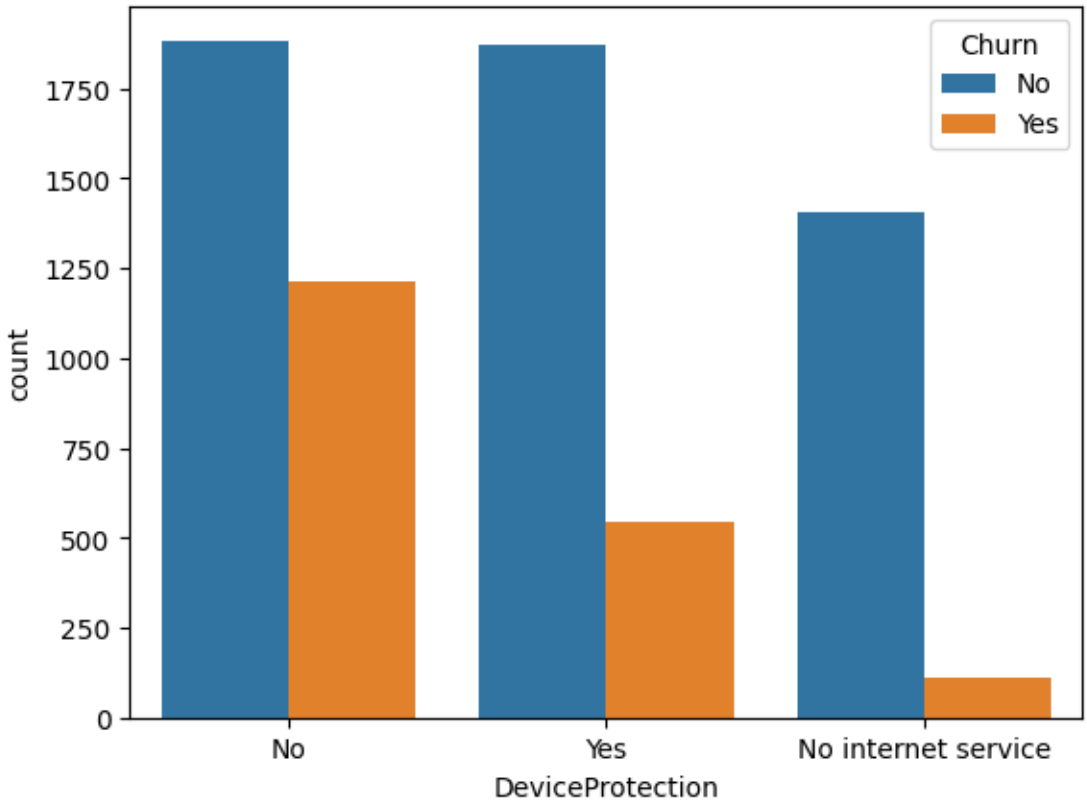

Churn
No
Yes
4000
3000
2000
1000
0
count
No
Yes
PhoneService

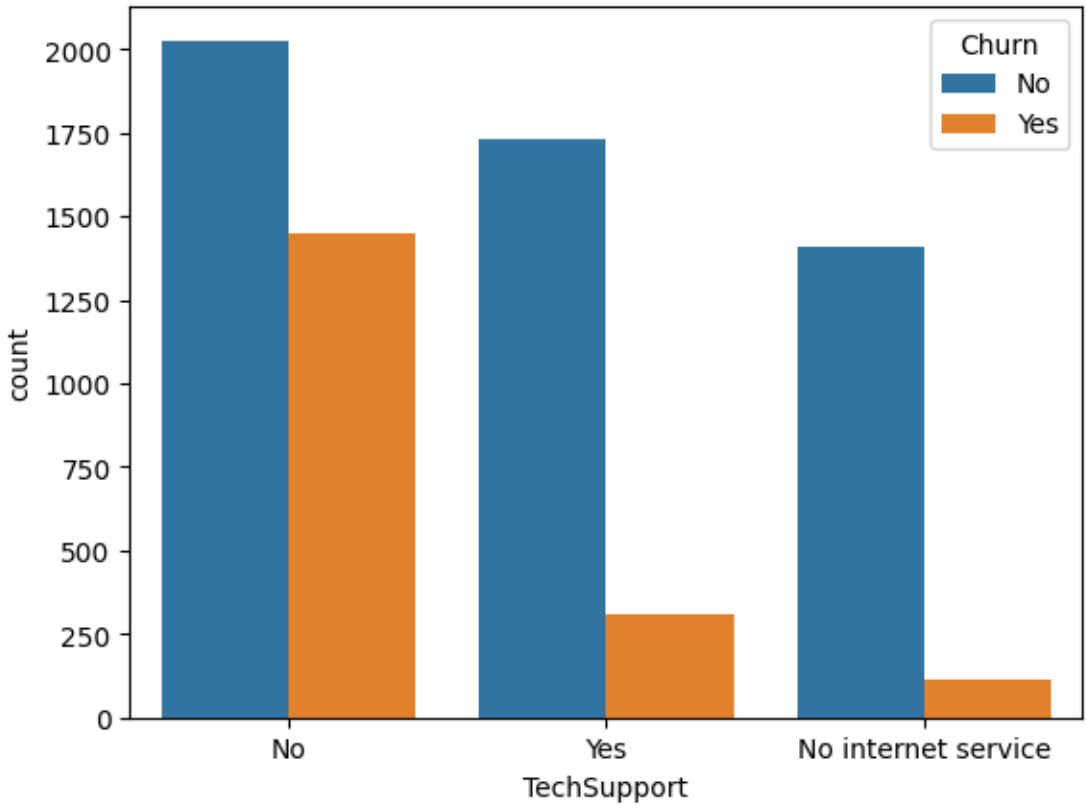

Churn
No
Yes
2500
2000
1500
1000
500
0
count
No phone service
No
Yes
MultipleLines

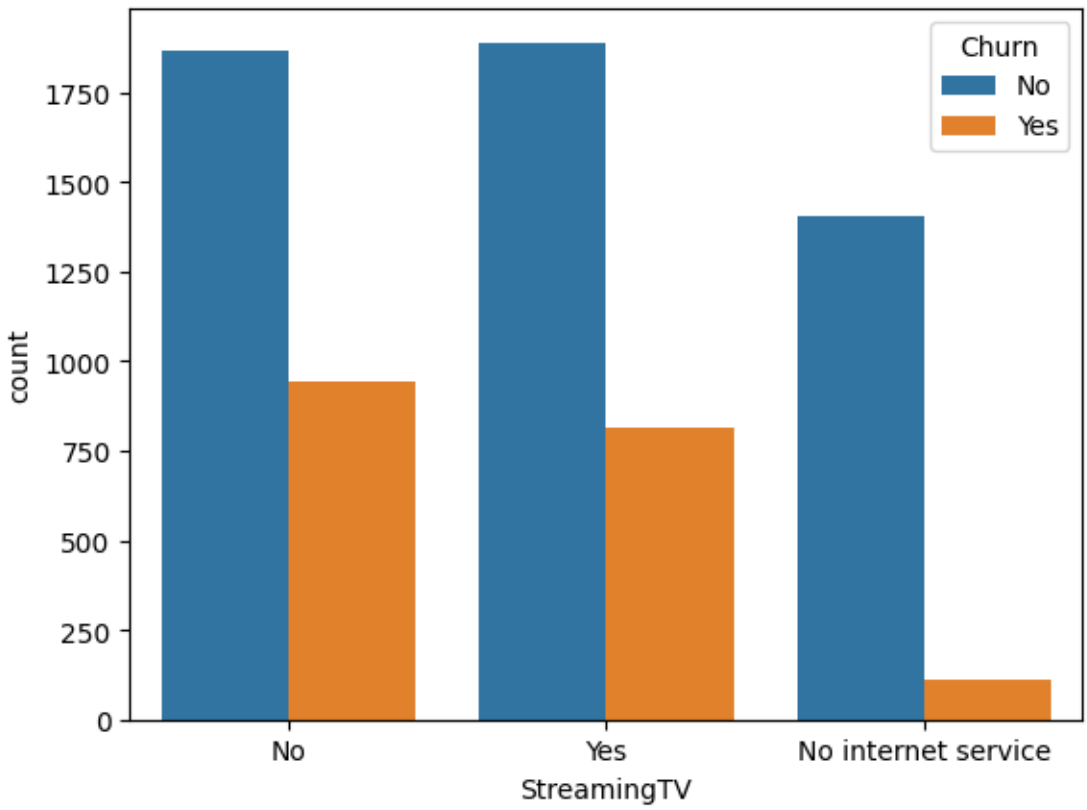

Churn
No
Yes
2000
1750
1500
1250
1000
750
500
250
0
count
DSL
Fiber optic
No
InternetService

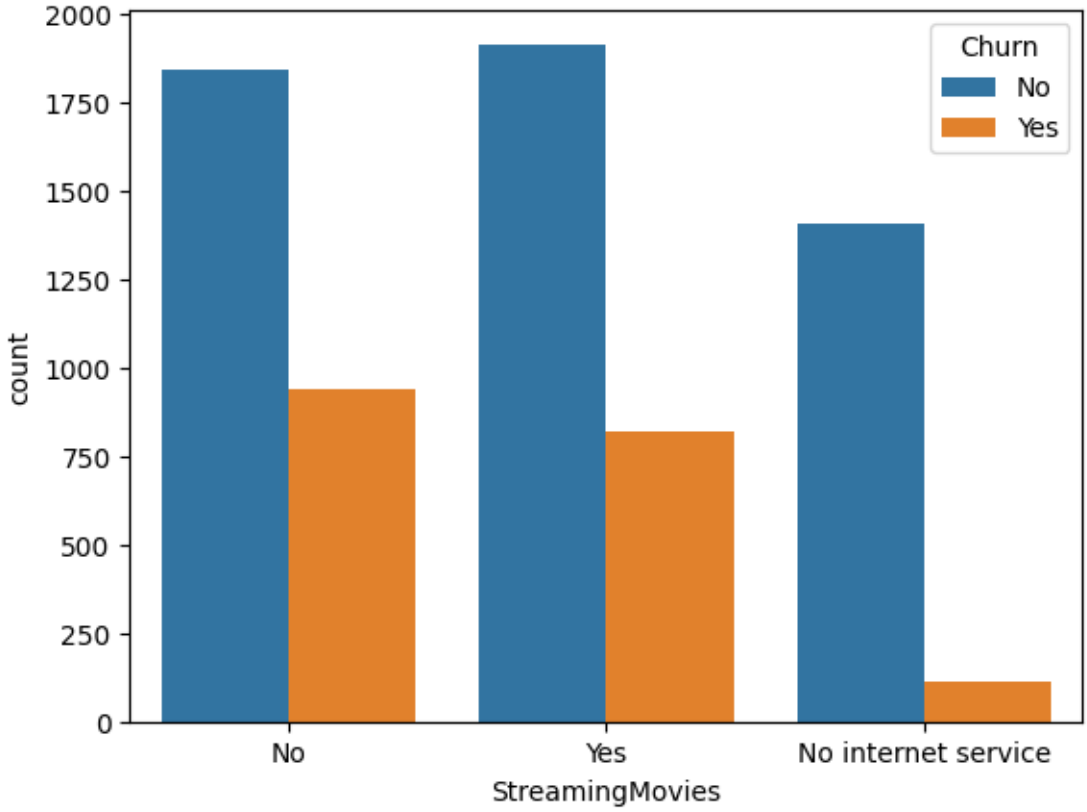

Churn
No
Yes
2000
1750
1500
1250
1000
750
500
250
0
count
No
Yes
No internet service
OnlineSecurity

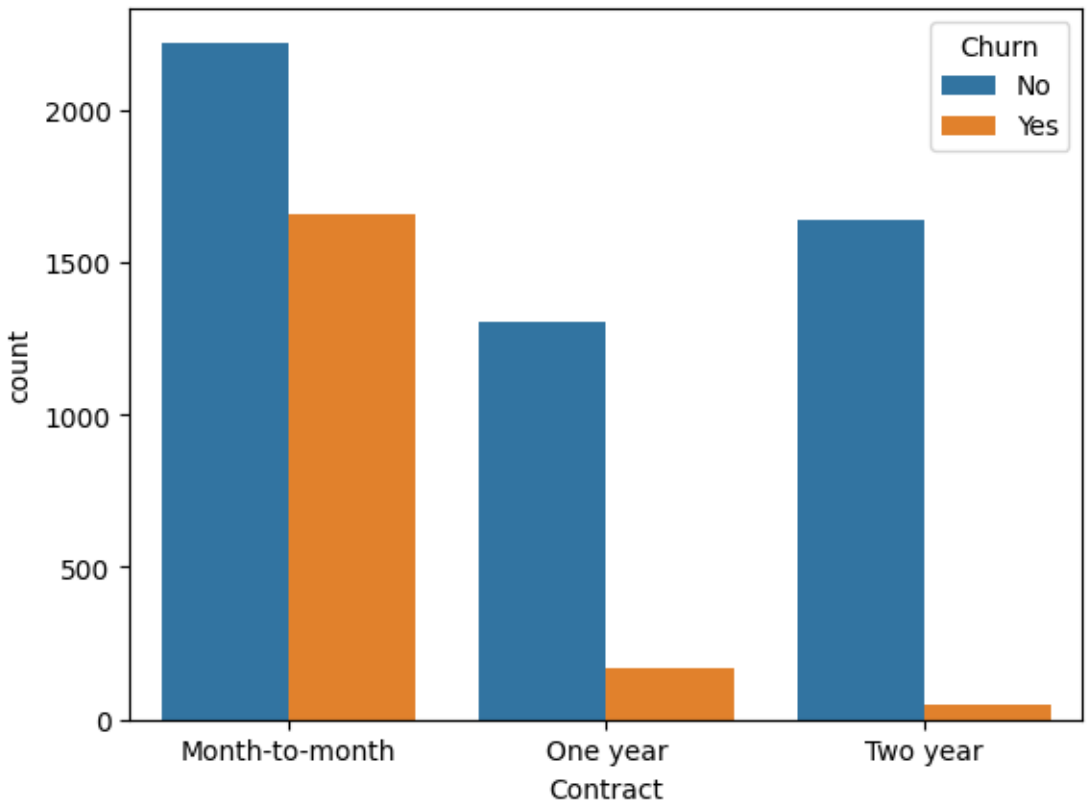

Churn
No
Yes
count
1750
1500
1250
1000
750
500
250
0
Yes
No
No internet service
OnlineBackup

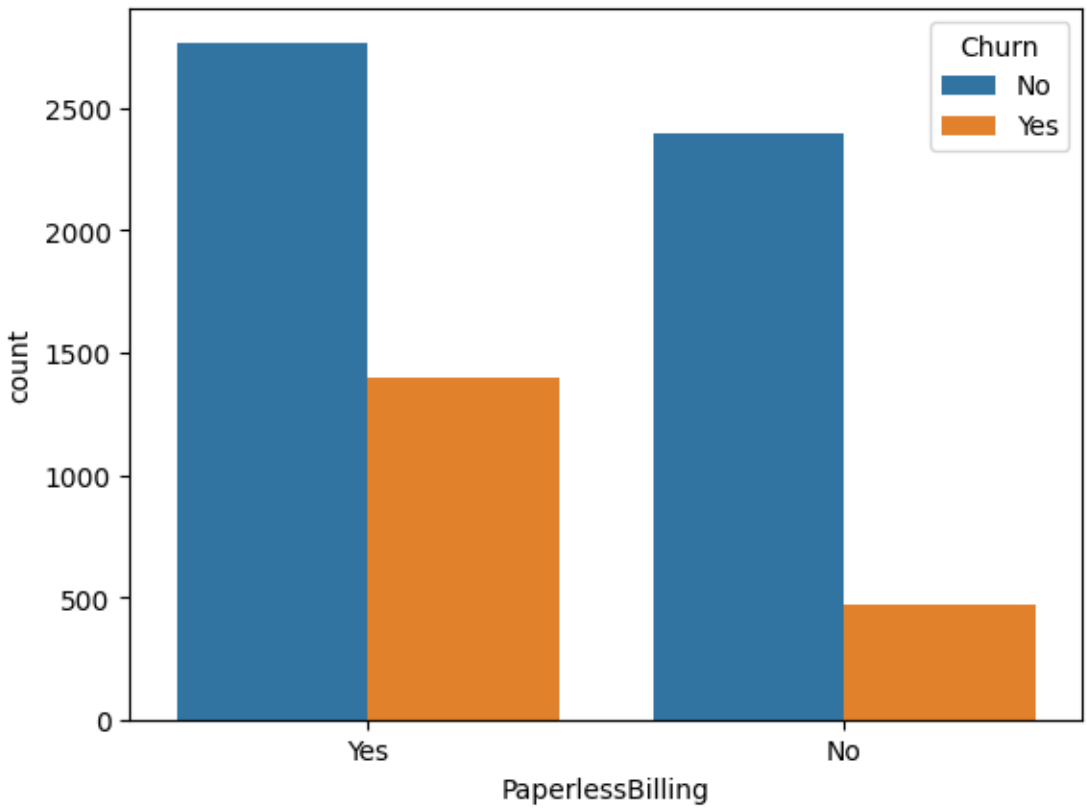

Churn
No
Yes
count
1750
1500
1250
1000
750
500
250
0
No
Yes
No internet service
DeviceProtection

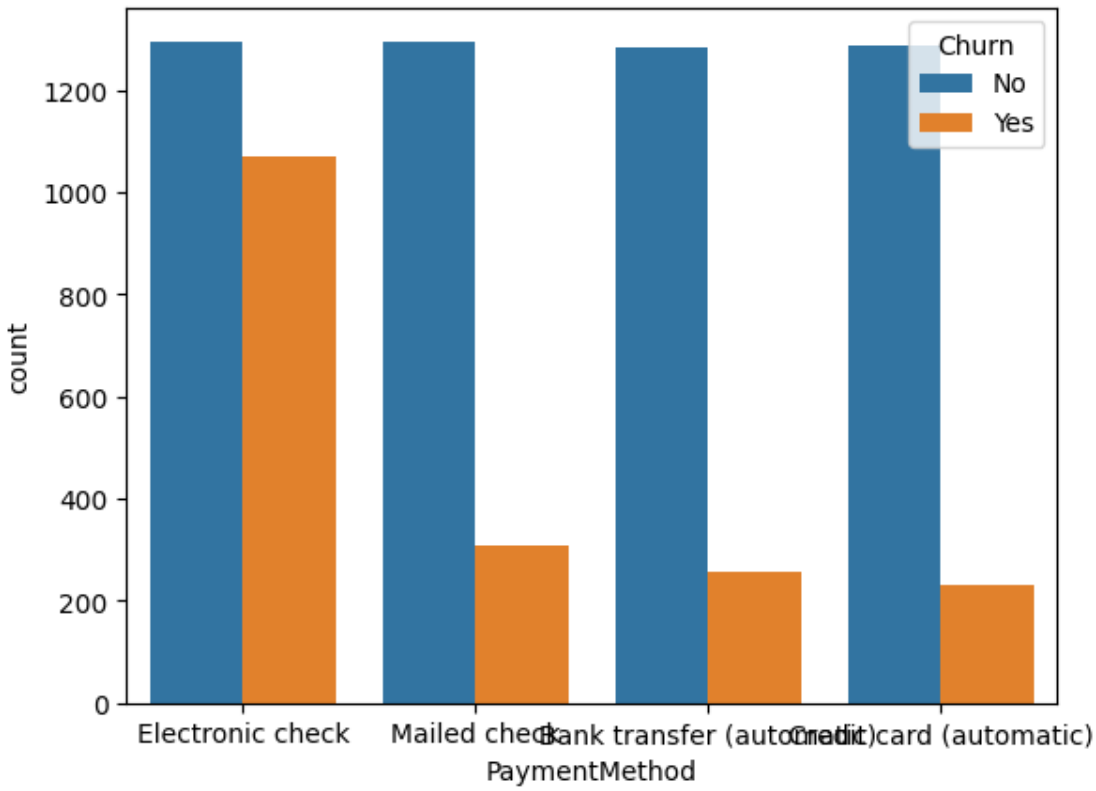

Churn
No
Yes
2000
1750
1500
1250
1000
750
500
250
0
count
No
Yes
No internet service
TechSupport

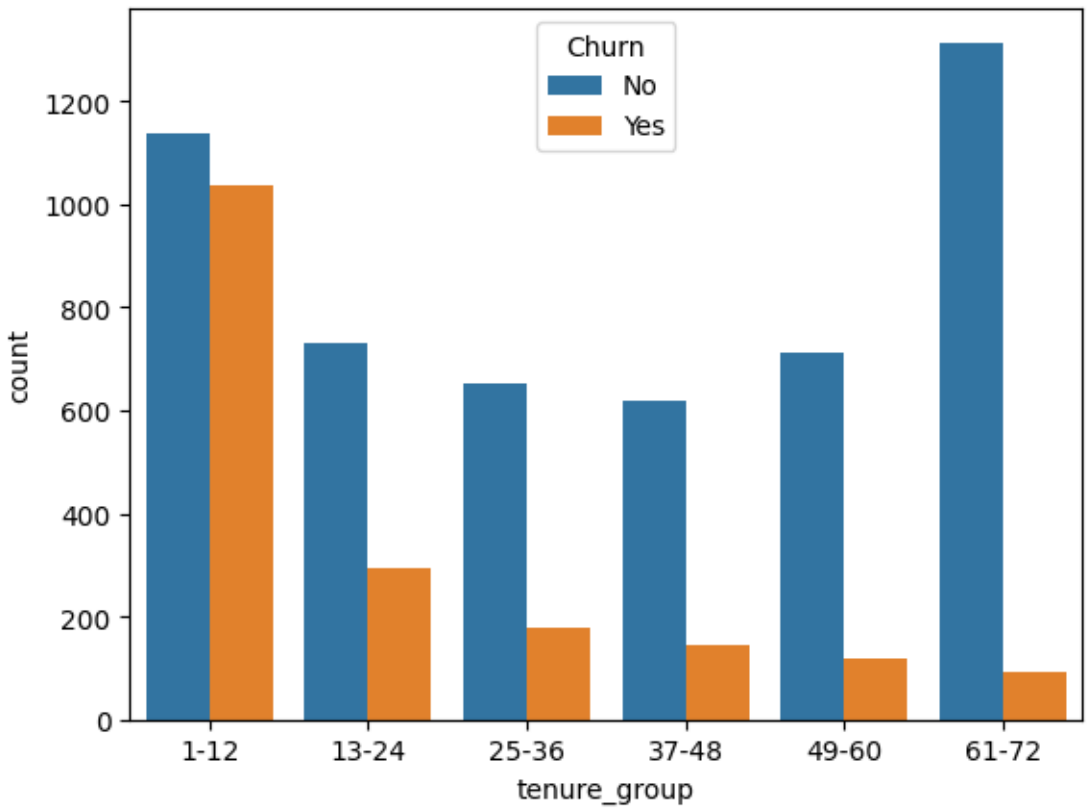

Churn
No
Yes
count
1750
1500
1250
1000
750
500
250
0
No
Yes
No internet service
StreamingTV


2000
1750
1500
1250
1000
750
500
250
0
count
Churn
No
Yes
No
Yes
No internet service
StreamingMovies


Churn
No
Yes
2000
1500
1000
500
0
count
Month-to-month
One year
Two year
Contract


Churn
No
Yes
2500
2000
1500
1000
500
0
count
Yes
No
PaperlessBilling


Churn
No
Yes
count
0
200
400
600
800
1000
1200
Electronic check
Mailed check
Bank transfer (automatic)
Credit card (automatic)
PaymentMethod



****2.convert the target variable 'churn' in a binary numeric variable i. yes=1; no=0****

[47]:
```
telco_data['Churn']=np.where(telco_data.Churn=='Yes',1,0)
```

[48]:
```
telco_data.head()
```

[48]:
```
   gender  SeniorCitizen Partner Dependents PhoneService     MultipleLines  \
0  Female              0     Yes         No           No  No phone service
1    Male              0      No         No          Yes                No
2    Male              0      No         No          Yes                No
3    Male              0      No         No           No  No phone service
4  Female              0      No         No          Yes                No

  InternetService OnlineSecurity OnlineBackup DeviceProtection TechSupport  \
0             DSL             No          Yes               No          No
1             DSL            Yes           No              Yes          No
2             DSL            Yes          Yes               No          No
3             DSL            Yes           No              Yes         Yes
4     Fiber optic             No           No               No          No

  StreamingTV StreamingMovies        Contract PaperlessBilling  \
```

```
0          No              No  Month-to-month              Yes
1          No              No        One year               No
2          No              No  Month-to-month              Yes
3          No              No        One year               No
4          No              No  Month-to-month              Yes

               PaymentMethod  MonthlyCharges  TotalCharges  Churn tenure_group
0           Electronic check           29.85         29.85      0         1-12
1               Mailed check           56.95       1889.50      0        25-36
2               Mailed check           53.85        108.15      1         1-12
3  Bank transfer (automatic)           42.30       1840.75      0        37-48
4           Electronic check           70.70        151.65      1         1-12
```

3. Converting all the categorical variables into dummy variables

[50]:
```
telco_data_dummies = pd.get_dummies(telco_data).astype(int)
telco_data_dummies.head()
```

[50]:
```
   SeniorCitizen  MonthlyCharges  TotalCharges  Churn  gender_Female  \
0              0              29            29      0              1
1              0              56          1889      0              0
2              0              53           108      1              0
3              0              42          1840      0              0
4              0              70           151      1              1

   gender_Male  Partner_No  Partner_Yes  Dependents_No  Dependents_Yes  …  \
0            0           0            1              1               0  …
1            1           1            0              1               0  …
2            1           1            0              1               0  …
3            1           1            0              1               0  …
4            0           1            0              1               0  …

   PaymentMethod_Bank transfer (automatic)  \
0                                        0
1                                        0
2                                        0
3                                        1
4                                        0

   PaymentMethod_Credit card (automatic)  PaymentMethod_Electronic check  \
0                                      0                               1
1                                      0                               0
2                                      0                               0
3                                      0                               0
4                                      0                               1

   PaymentMethod_Mailed check  tenure_group_1-12  tenure_group_13-24  \
```

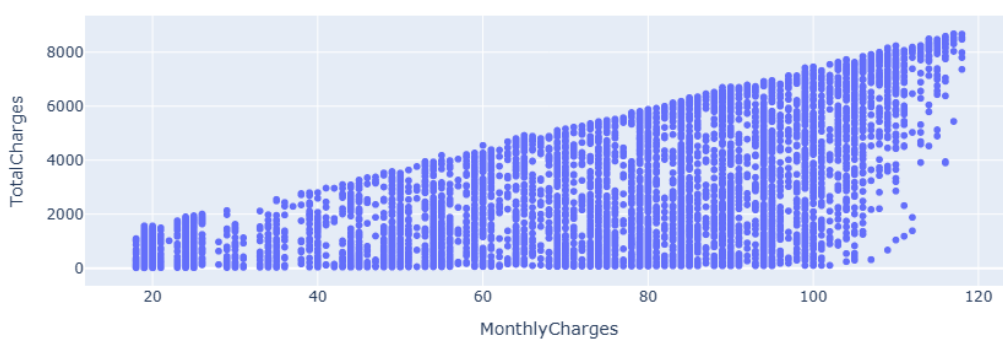
```
0                           0                   1                   0
1                           1                   0                   0
2                           1                   1                   0
3                           0                   0                   0
4                           0                   1                   0

   tenure_group_25-36  tenure_group_37-48  tenure_group_49-60  \
0                   0                   0                   0
1                   1                   0                   0
2                   0                   0                   0
3                   0                   1                   0
4                   0                   0                   0

   tenure_group_61-72
0                   0
1                   0
2                   0
3                   0
4                   0

[5 rows x 51 columns]
```

[51]:
```
telco_data_dummies.shape
```

[51]: (7032, 51)

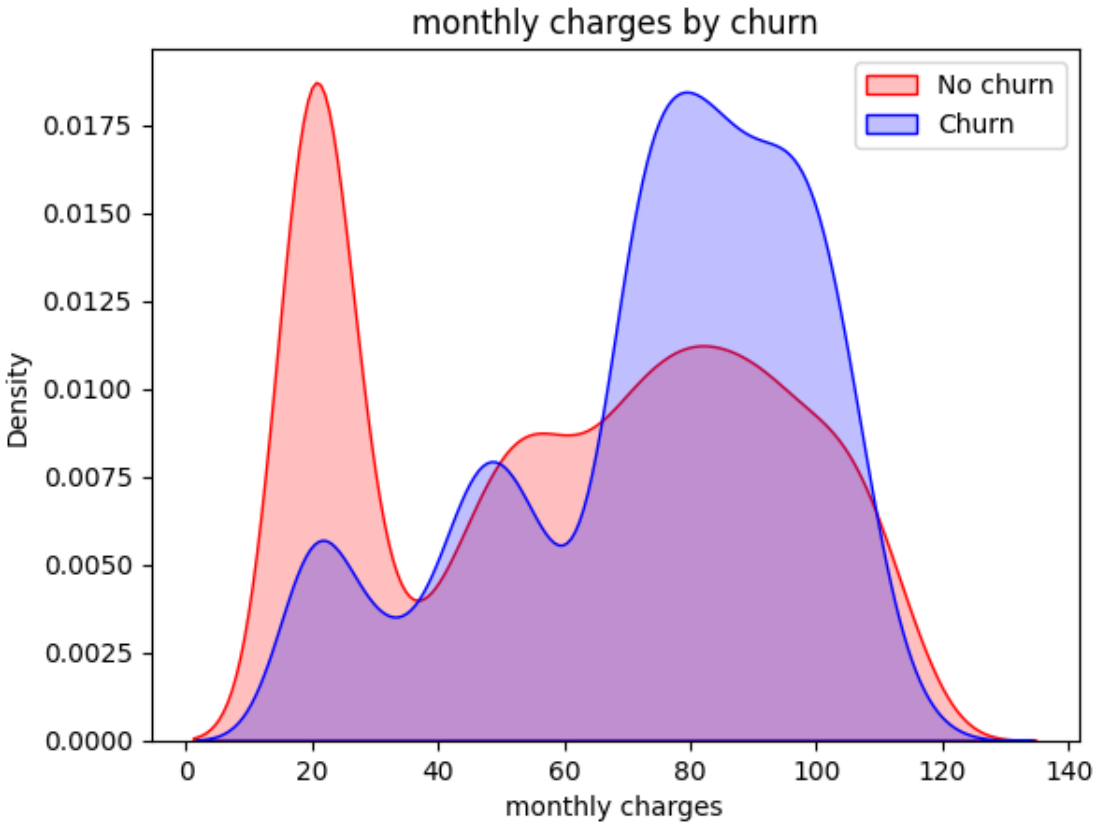
9. Relationship Between Monthly charges and Total Charges

[53]:
```
px.scatter(telco_data_dummies,x='MonthlyCharges', y='TotalCharges')
```



10 churn by monthly charges and total charges

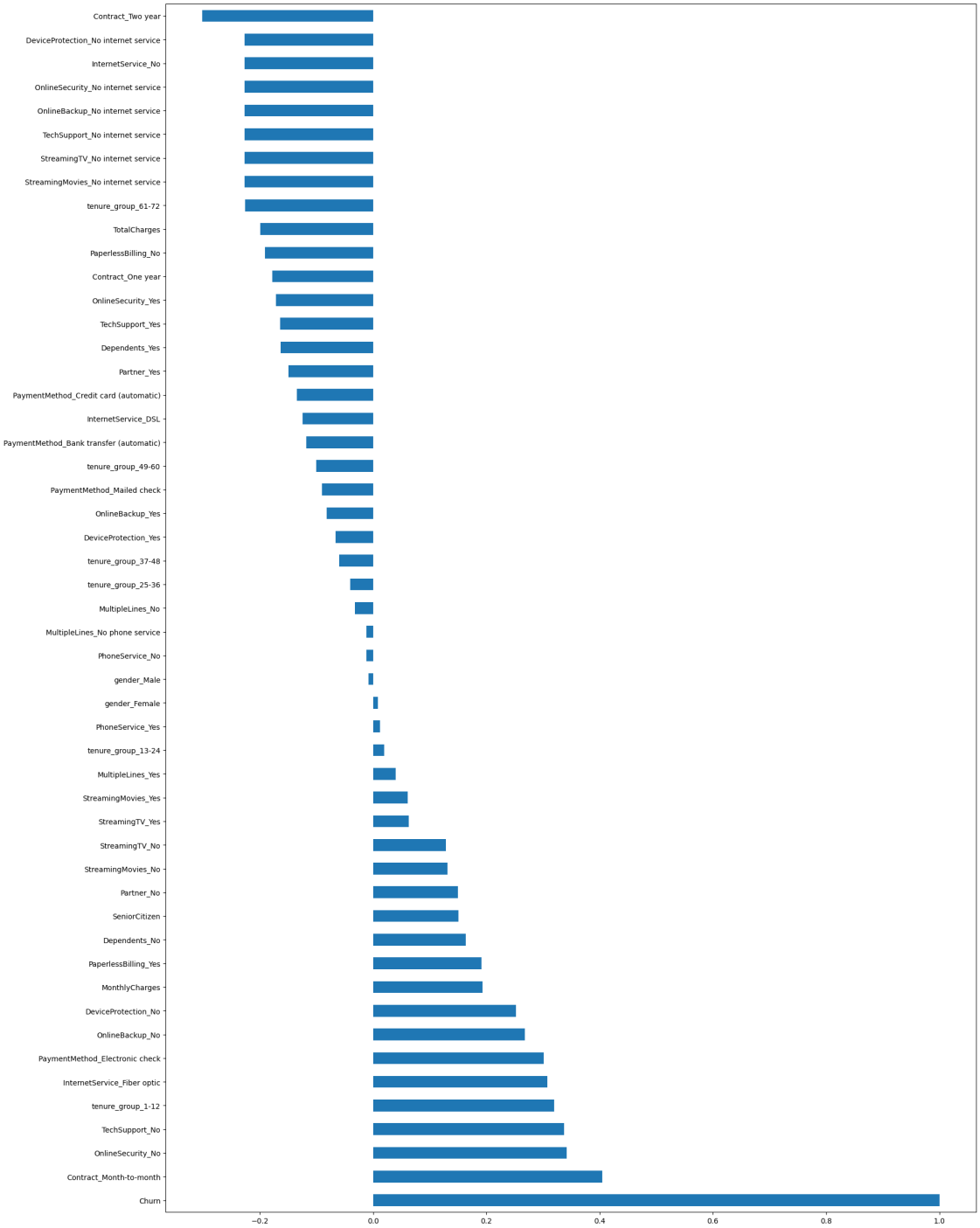
[55]:
```
mth = sns.kdeplot(telco_data_dummies.
 ↪MonthlyCharges[(telco_data_dummies['Churn']==0)], color='Red',fill=True)
mth=sns.kdeplot(telco_data_dummies.
 ↪MonthlyCharges[(telco_data_dummies['Churn']==1)],ax=mth,color='Blue',fill=True)
mth.legend(['No churn','Churn'],loc='upper right')
mth.set_ylabel('Density')
mth.set_xlabel("monthly charges")
mth.set_title('monthly charges by churn')
plt.show()
```



Insigth: Churn is high when monthly charges are high

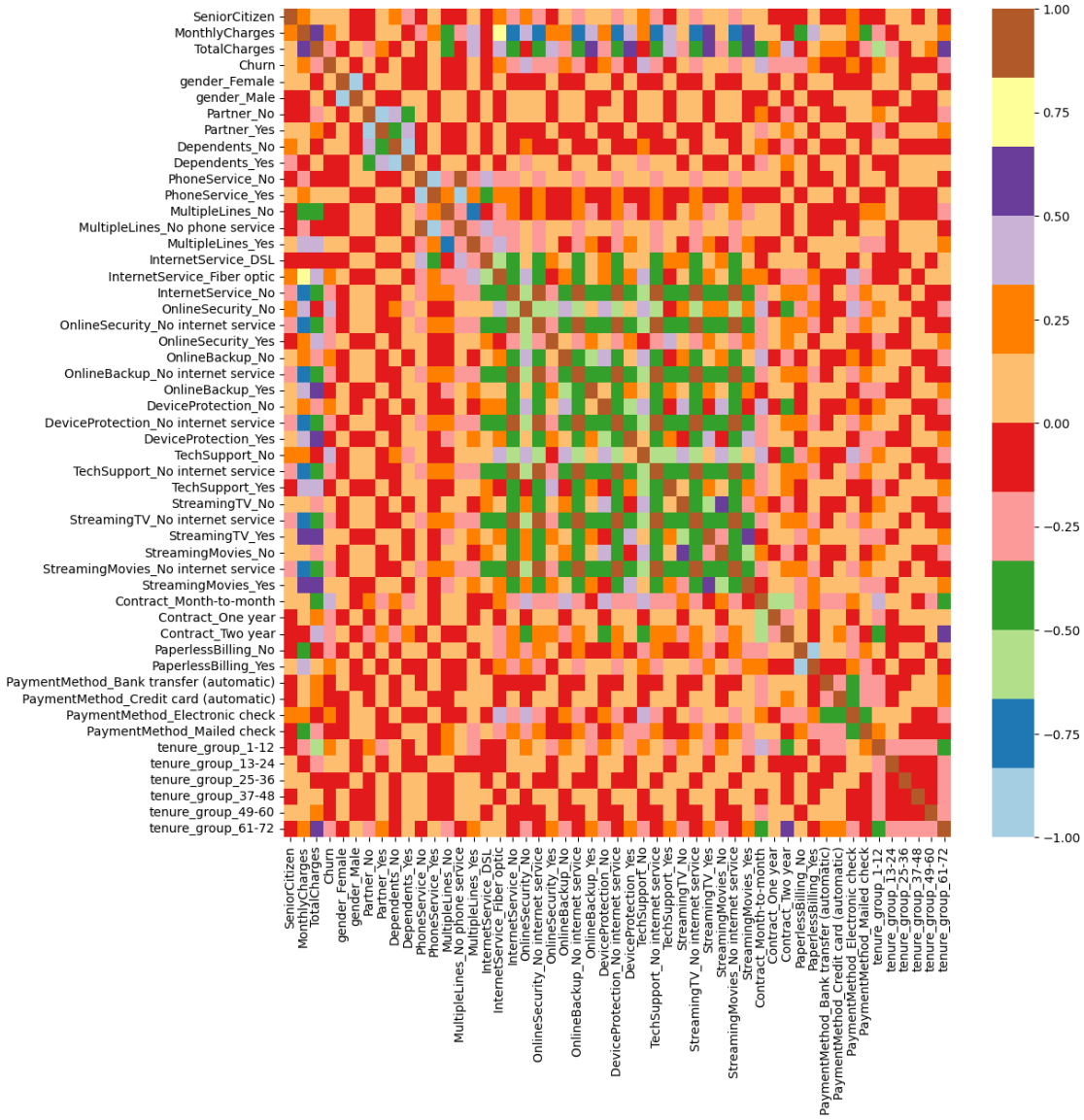
11. Build a correlation of all predictors with Churn

[58]:
```
plt.figure(figsize=(20,30))
telco_data_dummies.corr()['Churn'].sort_values(ascending=False).
 ↪plot(kind='barh')
plt.show()
```



Derived Insigths - High churn seen in case of month contracts, no onliine security,no tech support, First year of subscription and Fibre optics internet - Low churn is seen in case of long term contracts , Subscriptions without internet servies and the customers engaged for 5+ years - Factors like Gender,Availability of PhoneService and multiple lines have almost NO impact on churn

[60]:
```
plt.figure(figsize=(12,12))
sns.heatmap(telco_data_dummies.corr(),cmap='Paired')
plt.show()
```



[65]:
```
telco_data_dummies.to_csv('F:\\data sets\\tel_churn.csv')
```

2.0.2 ML Implmentation

[102]:
```
import pandas as pd
import numpy as np
from sklearn import metrics
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import recall_score,confusion_matrix,classification_report
from sklearn.tree import DecisionTreeClassifier
from imblearn.combine import SMOTEENN
```

[103]:
```
df = pd.read_csv("F:\\data sets\\tel_churn.csv")
```

[104]:
```
df.drop(columns=['Unnamed: 0'],inplace=True)
```

[105]:
```
df.head()
```

[105]:
```
   SeniorCitizen  MonthlyCharges  TotalCharges  Churn  gender_Female  \
0              0              29            29      0              1
1              0              56          1889      0              0
2              0              53           108      1              0
3              0              42          1840      0              0
4              0              70           151      1              1

   gender_Male  Partner_No  Partner_Yes  Dependents_No  Dependents_Yes  …  \
0            0           0            1              1               0  …
1            1           1            0              1               0  …
2            1           1            0              1               0  …
3            1           1            0              1               0  …
4            0           1            0              1               0  …

   PaymentMethod_Bank transfer (automatic)  \
0                                        0
1                                        0
2                                        0
3                                        1
4                                        0

   PaymentMethod_Credit card (automatic)  PaymentMethod_Electronic check  \
0                                      0                                1
1                                      0                                0
2                                      0                                0
3                                      0                                0
4                                      0                                1

   PaymentMethod_Mailed check  tenure_group_1-12  tenure_group_13-24  \
0                           0                  1                   0
1                           1                  0                   0
2                           1                  1                   0
3                           0                  0                   0
4                           0                  1                   0

   tenure_group_25-36  tenure_group_37-48  tenure_group_49-60  \
```

```
0                    0                    0                    0
1                    1                    0                    0
2                    0                    0                    0
3                    0                    1                    0
4                    0                    0                    0

   tenure_group_61-72
0                    0
1                    0
2                    0
3                    0
4                    0

[5 rows x 51 columns]
```

[166]:
```python
# creating X and y variable
X = df.drop('Churn',axis=1)
y = df['Churn']
```

[168]:
```python
x_train,x_test,y_train,y_test = train_test_split(X,y,test_size=0.2)
```

2.0.3 Decision Tree Classifier

[171]:
```python
model = DecisionTreeClassifier(criterion='gini',random_state=100,␣
↪max_depth=6,min_samples_leaf=8 )
```

[173]:
```python
model.fit(x_train,y_train)
```

[173]:
```
DecisionTreeClassifier(max_depth=6, min_samples_leaf=8, random_state=100)
```

[175]:
```python
y_pred=model.predict(x_test)
y_pred
```

[175]:
```
array([0, 0, 0, …, 0, 0, 0], dtype=int64)
```

[177]:
```python
from sklearn.metrics import accuracy_score, confusion_matrix
accuracy = accuracy_score(y_test, y_pred)
confusion_matrix  = confusion_matrix(y_test, y_pred)
print(accuracy*100)
print(confusion_matrix)
```

```
79.45984363894812
[[956  89]
 [200 162]]
```

[179]:
```python
print(classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           0       0.83      0.91      0.87      1045
           1       0.65      0.45      0.53       362

    accuracy                           0.79      1407
   macro avg       0.74      0.68      0.70      1407
weighted avg       0.78      0.79      0.78      1407
```

- we can see that the precision,recall,f1-score are very low for churn->1
- This happens due to imblanced data
- To balance the data(upsampling) we use SMOTEENN() technique

[182]:
```
sm = SMOTEENN()
x_resampled,y_resampled = sm.fit_resample(X,y)
```

[183]:
```
xr_train,xr_test,yr_train,yr_test =␣
↪train_test_split(x_resampled,y_resampled,test_size=0.2)
```

[184]:
```
model_dt_smote =␣
↪DecisionTreeClassifier(criterion='gini',random_state=100,max_depth=6,min_samples_leaf=8)
```

[185]:
```
model_dt_smote.fit(xr_train,yr_train)
```

[185]:
```
DecisionTreeClassifier(max_depth=6, min_samples_leaf=8, random_state=100)
```

[190]:
```
y_pred_smote = model_dt_smote.predict(xr_test)
```

[192]:
```
print(classification_report(yr_test,y_pred_smote))
```

```
              precision    recall  f1-score   support

           0       0.94      0.91      0.93       522
           1       0.93      0.96      0.94       636

    accuracy                           0.94      1158
   macro avg       0.94      0.93      0.94      1158
weighted avg       0.94      0.94      0.94      1158
```

2.0.4 Random Forest Classifier

[195]:
```
from sklearn.ensemble import RandomForestClassifier
```

- **Using Imblanced Data**

[197]:
```
model_rf =␣
↪RandomForestClassifier(n_estimators=100,random_state=100,max_depth=6,min_samples_leaf=8)
model_rf.fit(x_train,y_train)
y_pred_rf=model_rf.predict(x_test)
```

[199]:
```
print(classification_report(y_test,y_pred_rf,labels=[0,1]))
```

```
              precision    recall  f1-score   support

           0       0.83      0.92      0.88      1045
           1       0.67      0.47      0.56       362

    accuracy                           0.81      1407
   macro avg       0.75      0.70      0.72      1407
weighted avg       0.79      0.81      0.79      1407
```

- **Using Balanced Data**

[207]:
```
model_rff =␣
↪RandomForestClassifier(n_estimators=100,random_state=100,max_depth=6,min_samples_leaf=8)
model_rff.fit(xr_train,yr_train)
y_pred_rff=model_rff.predict(xr_test)
```

[209]:
```
print(classification_report(yr_test,y_pred_rff))
```

```
              precision    recall  f1-score   support

           0       0.96      0.92      0.94       522
           1       0.93      0.97      0.95       636

    accuracy                           0.94      1158
   macro avg       0.95      0.94      0.94      1158
weighted avg       0.95      0.94      0.94      1158
```

- **Saving the model**

[215]:
```
import pickle
```

[221]:
```
filename = 'telco_churn_model.sav'
```

[223]:
```
pickle.dump(model_rff,open(filename,'wb'))
```

- **Using the saved model to get Score**

[225]:
```
load_model = pickle.load(open(filename,'rb'))
```

[227]:
```
load_model.score(xr_test,yr_test)
```

[227]: 0.9447322970639033

[ ]: